

SAE : Application Répartie

Consignes pour les présentations de SAE :

Les présentations et démos de la SAE "Application Répartie" auront lieu le **mardi 16 juin en salle 125 ou 127** selon votre groupe.

L'ordre et l'heure de passage seront mis en ligne sur la page ARCHE. Merci de renseigner la composition des équipes sur le pad dédié.

Évaluation & Modalités de passage

Pour la présentation, chaque équipe disposera de conditions spécifiques :

- Vous devez utiliser le vidéo-projecteur.
- Chaque équipe aura **10 min** pour présenter son travail.
- Votre site web devra contenir un onglet qui illustre l'architecture de votre application et ses différents composants par des schémas.
- Tous les membres de l'équipe devront participer à la démonstration.
- Essayez de scénariser votre présentation.

Points évalués :

- Structuration du code, choix des classes/packages.
- Robustesse de l'application face aux problèmes réseaux.
- Ne pas coder les adresses IP ou les URL en dur dans le code.
- Gestion des dépendances (Librairies externes) avec Maven ou npm.
- Compilation automatique avec Maven ou Ant.
- Javadoc de l'API publique.
- Documentation pour déployer et exécuter.
- Exécution sur les machines de l'IUT.
- Utilisation du serveur Web de webetu.
- Déploiement automatique sur les machines de la salle.

Considérations techniques importantes :

- Privilégier l'exécution sous **Linux** car la démo se fera sous Linux.
- Il n'y a pas de Java sur la machine `webetu` (il ne servira qu'à héberger votre site).
- La version de Java sur charlemagne n'est pas la même (plus vieille) que celle sur les machines du réseau.
- Le SGBD sur charlemagne est **Oracle**. L'utilisation de MariaDB sur webetu sera plus difficile.

Introduction & Objectifs

Cette SAE concerne les enseignements suivants :

- **R4.01** : Architecture logicielle
- **R4.02** : Qualité de dév. / tests
- **R4.03** : Qualité et au-delà du relationnel
- **R4.10** : Complément web

Le travail se fera par groupe de **4 étudiants**.

L'objectif est de proposer une application répartie qui permettra d'afficher, dans un navigateur, sur une carte **Leaflet**, des informations hétérogènes sur Nancy (informations vous appartenant ou pas, transitant parfois par un proxy, qui fera la passerelle entre votre navigateur et des services).

Votre projet devra combiner les différents modules proposés ci-dessous, accessibles depuis un site web hébergé sur `webetu`.

Votre site devra proposer un onglet "**Compte rendu**", contenant le compte rendu de votre travail :

- Un schéma complet de votre architecture, avec les technologies employées et échangées.
- Ce qui a été fait.
- Les informations nécessaires à l'utilisation de votre projet.
- ...et tout ce qui est utile à la compréhension de votre travail.
- Le lien vers votre dépôt Git avec tous les codes sources.

À la fin de la SAE, vous devrez effectuer une démonstration. Pour cette démonstration, vous exécuterez vos services sur des machines différentes de la salle de soutenance.

1. Un service RMI qui interroge une base de données

Proposer un service RMI (et un client de test) permettant d'accéder à votre base de données de restaurants, base que vous devez enrichir et étendre.

- Ajouter à votre base des entrées de restaurants avec leur nom, leur adresse, et leurs coordonnées GPS (situer les à Nancy).
- Le service RMI devra pouvoir :
 - S'inscrire sur un service central.
 - Récupérer toutes les coordonnées de tous les restaurants de votre base de données.
 - Réserver une table dans un restaurant en renseignant nom, prénom, nombre de convives, et coordonnées téléphoniques de la réservation.
- Les réponses du service devront se faire au format **JSON**.

2. Des données ouvertes (Open Data)

Données partout...

De plus en plus de données dites ouvertes sont disponibles depuis le web, fournies par différents organismes, souvent d'État pour lesquels la diffusion est parfois obligatoire. (Quelles informations propose `data.gouv.fr` ?)

Visualisation des données

Créer une carte lisible depuis un navigateur centrée sur Nancy. Ajouter sur votre carte, pour toutes les stations Vélib :

- L'adresse de la station Vélib.
- Le nombre de vélos disponibles sur la station.
- Le nombre de places de parkings libres sur la station.

À noter que de nombreuses autres données sont disponibles et peuvent être utilisées.

3. Un service qui interroge les données bloquées

Des données bloquées sur le client

Certaines données posent problème. Essayez de récupérer, depuis votre site, ces données en utilisant la console (cachée derrière la touche F12) de votre navigateur, identifiez le problème.

Un service qui interroge des données ouvertes

En utilisant la classe `HttpClient` (et en particulier le *Synchronous Example*) de Java, proposer un service permettant de récupérer ces données.

- Penser à tester la bonne réception (code d'envoi) et le format des données.

- *Attention* : depuis les machines de l'IUT, vous devez renseigner un proxy pour accéder à l'extérieur. Vous trouverez les paramètres de ce proxy dans la documentation informatique de l'IUT ; vous devrez le retrouver dans la configuration par défaut de vos navigateurs (qui sont des clients Http).
- Vous pouvez utiliser ces classes, cf. leur documentation.
- Penser à gérer les erreurs possibles dues à l'utilisation d'un réseau.

4. Un proxy qui interroge des services

À l'aide de la classe `HttpServer`, proposer un proxy entre votre site et les données bloquées.

Ce proxy devra proposer un service RMI donnant accès à vos services RMI de restaurants et aux "incidents de circulation" liés aux travaux dans la Métropole du Grand Nancy.

En passant par ce proxy, ajouter sur votre carte :

- Les lieux des incidents prévus, avec leur adresse exacte, leur cause et les dates d'incidence.
- Votre liste de restaurants.
- Il doit être possible de réserver une table depuis la marque d'un restaurant.

Question de réflexion : Est-ce responsable d'ainsi contourner la politique de sécurité de votre navigateur ?